



Proyecto Gimnasio

CRUD completo con Spring Boot 3.5.15 + Angular 21

Java 25 (Temurin)

Spring Boot 3.5.15

Spring Data JPA

MySQL 8.4

Angular 21.2.15

Bootstrap 5.3.8

Swagger / OpenAPI

Ejercicio para el alumnado · Academia Arelance
backGimnasio | frontGimnasio



Enunciado del ejercicio

Vas a construir, de principio a fin, una aplicación de gestión para un **gimnasio**, reutilizando todo lo aprendido en el proyecto *crudalumnos*. El objetivo es que repliques la arquitectura **backend (Spring Boot) + frontend (Angular)** sobre un dominio nuevo.

A entregar

Dos proyectos dentro de la carpeta **Gimnasio** :

- **backGimnasio** → API REST con Spring Boot 3.5.15 (Java 25, Maven, JPA, MySQL 8.4, Swagger).
- **frontGimnasio** → SPA con Angular 21 + Bootstrap 5, consumiendo la API anterior.

Entidades del dominio

Entidad	Tabla	Descripción
Socio	socios	Personas inscritas en el gimnasio. PK = NIF.
Monitor	monitores	Profesionales que imparten las actividades. PK autonómica.
Actividad	actividades	Clases ofertadas (spinning, yoga, etc.). PK autonómica.
Suscripcion	suscripciones	Tabla intermedia ternaria: une un Socio, un Monitor y una Actividad.

El reto de la tabla ternaria

A diferencia de **alumnos_asignaturas** (que unía 2 entidades), **suscripciones** une **3**: un mismo socio se suscribe a una actividad *impartida por* un monitor concreto. Igual que en los apuntes, **NO uses @ManyToMany** : define la entidad intermedia a mano con tres **@ManyToOne** (uno por cada entidad).

1. Modelo de Base de Datos

Este es el diagrama Entidad-Relación que debes implementar con anotaciones JPA. Cada entidad tiene su color; respétalo también en los mockups del frontend.

■ Socio ■ Monitor ■ Actividad ■ Suscripción (puente ternario)

🧑 socios		
varchar	nif	PK
varchar	nombre	
int	edad	
varchar	email	
boolean	activo	

🏆 monitores		
int	id	PK
varchar	nombre	
varchar	especialidad	
varchar	email	

🏃 actividades		
int	id	PK
varchar	denominacion	
int	plazas	
decimal	precioMes	

↑ tiene ↑ imparte ↑ contiene ↑

📝 suscripciones		
bigint	id	PK
varchar	socio_nif	FK → socios
int	monitor_id	FK → monitores
int	actividad_id	FK → actividades
date	fechaAlta	
boolean	activa	

📝 Restricción de unicidad

En `suscripciones` añade un `@UniqueConstraint` sobre `(socio_nif, monitor_id, actividad_id)`: un socio no debe suscribirse dos veces a la misma actividad con el mismo monitor. Igual que hicimos con `(alumno_nif, asignatura_id)`.

💡 Pista de mapeo JPA

En `Suscripcion.java` tendrás tres `@ManyToOne` con `@JsonIgnore` en el lado del socio y del monitor para romper la recursión en la serialización (como en `AlumnoAsignatura`). En `Socio`, `Monitor` y `Actividad` pon el `@OneToMany(mappedBy=...)` con `@JsonIgnore`.

2. Mockups del frontend

Estas son las cinco pantallas que debe tener `frontGimnasio` . Cada pantalla mapea contra los endpoints REST que aparecen justo debajo de ella. **Reproduce los colores**: son la guía visual del dominio.

2.1 Pantalla de Socios

Gestión de Socios

/socios

248

Socios totales

214

Activos

34

De baja

Buscar por nombre o NIF...

Estado ▾

+ Nuevo socio

NIF	Nombre	Edad	Email	Estado	Acciones
11111111A	Lucía Romero	28	lucia@mail.com	Activo	 
22222222B	Marcos Gil	34	marcos@mail.com	Activo	 
33333333C	Sara Núñez	22	sara@mail.com	Baja	 
44444444D	Iván Torres	41	ivan@mail.com	Activo	 

Mostrando 1–4 de 248 < 1 2 >

Verbo	Endpoint	Qué hace
GET	/api/socios/consultar	Listar todos los socios (tabla)
GET	/api/socios/consultar/{nif}	Ver un socio por su NIF
GET	/api/socios/consultar/nombre?nombre=	Buscar por nombre (contiene)
POST	/api/socios/crear	Alta de socio (JSON)
PUT	/api/socios/actualizar/{nif}	Editar socio
DELETE	/api/socios/borrar/{nif}	Baja de socio

2.2 🏆 Pantalla de Monitores

 **Gestión de Monitores**

/monitores

12
Monitores

7
Especialidades

31
Clases asignadas

Especialidad ▾

+ Nuevo monitor

ID	Nombre	Especialidad	Email	Acciones
1	Diego Vela	Spinning	diego@gym.com	 
2	Nora Ferri	Yoga	nora@gym.com	 
3	Hugo Sanz	CrossFit	hugo@gym.com	 
4	Aitana Roca	Pilates	aitana@gym.com	 

Mostrando 1–4 de 12

Verbo	Endpoint	Qué hace
GET	/api/monitores/consultar	Listar todos los monitores
GET	/api/monitores/consultar/{id}	Ver monitor por ID
GET	/api/monitores/consultar/especialidad?valor=	Filtrar por especialidad
POST	/api/monitores/crear	Alta de monitor (JSON)
PUT	/api/monitores/actualizar/{id}	Editar monitor
DELETE	/api/monitores/borrar/{id}	Eliminar monitor

2.3 🏃 Pantalla de Actividades

 **Gestión de Actividades**

/actividades

9
Actividades

156
Plazas totales

29,90€
Precio medio/mes

+ Nueva actividad

ID	Denominación	Plazas	Precio/mes	Acciones
1	Spinning	20	34,90 €	 
2	Yoga	15	27,50 €	 
3	CrossFit	18	39,90 €	 
4	Pilates	12	29,90 €	 

Mostrando 1–4 de 9

Verbo	Endpoint	Qué hace
GET	/api/actividades/consultar	Listar todas las actividades
GET	/api/actividades/consultar/{id}	Ver actividad por ID
POST	/api/actividades/crear	Alta de actividad (JSON)
PUT	/api/actividades/actualizar/{id}	Editar actividad
DELETE	/api/actividades/borrar/{id}	Eliminar actividad

2.4 Pantalla de Suscripciones (la tabla ternaria)

 **Gestión de Suscripciones** /suscripciones

312
Suscripciones

287
Activas

1,3
Media/socio

Nueva suscripción

SOCIO

MONITOR

Lucía Romero (11111111A) ▼

Diego Vela — Spinning ▼

ACTIVIDAD

FECHA ALTA

Spinning ▼

15/09/2025 

Guardar suscripción

ID	Socio	Monitor	Actividad	Alta	Estado	
1	Lucía Romero	Diego Vela	Spinning	15/09/25	Activa	×
2	Marcos Gil	Nora Ferri	Yoga	20/09/25	Activa	×
3	Iván Torres	Hugo Sanz	CrossFit	01/10/25	Pausada	×

Mostrando 1–3 de 312

Verbo	Endpoint	Qué hace
GET	/api/suscripciones/consultar	Listar todas (con socio+monitor+actividad)
GET	/api/suscripciones/socio/{nif}	Suscripciones de un socio
GET	/api/suscripciones/actividad/{id}	Socios suscritos a una actividad
POST	/api/suscripciones/crear/{nif}/{idMonitor}/{idActividad}	Crear suscripción ternaria
DELETE	/api/suscripciones/borrar/{id}	Eliminar suscripción

JPQL para la tabla ternaria

El listado debe mostrar el nombre del socio, del monitor y de la actividad a la vez. Como cruza 3 tablas, necesitarás `@Query` con JPQL navegando por `s.socio.nombre`, `s.monitor.nombre` y `s.actividad.denominacion`. Plántate un `JOIN FETCH` de las tres relaciones para evitar el problema N+1.

2.5 📊 Pantalla de Gráficos y Mantenimiento



Verbo	Endpoint	Qué hace
GET	/api/socios/contar	KPI de socios
GET	/api/monitores/contar	KPI de monitores
GET	/api/actividades/contar	KPI de actividades
GET	/api/suscripciones/estadisticas/actividad	Datos del gráfico por actividad (GROUP BY)
GET	/api/suscripciones/estadisticas/monitor	Datos del gráfico por monitor (GROUP BY)
DELETE	/api/suscripciones/borrar/todo	Vaciar tabla (mantenimiento)

💡 Para los gráficos

En esta primera versión basta con devolver los conteos como JSON y pintar las barras con CSS/Bootstrap, igual que en este mockup. Más adelante podréis sustituirlas por un microservicio Python (FastAPI + matplotlib), como en el dashboard de *crudalumnos*.

3. Pasos sugeridos y criterios de evaluación

Orden de trabajo recomendado

1. Crear `backGimnasio` con Spring Initializr (Maven, Java 25, deps: Web, JPA, MySQL, Lombok, DevTools).
2. Crear la BBDD `gimnasio` en MySQL y configurar `application.properties` (`ddl-auto=update`).
3. Modelar las 4 entidades JPA respetando el diagrama (¡tabla ternaria a mano!).
4. Crear los 4 repositorios `JpaRepository` y un `CommandLineRunner` con datos demo.
5. Añadir Swagger y construir los controladores CRUD de las 4 entidades.
6. Crear `frontGimnasio` con Angular 21 + Bootstrap 5 y configurar CORS en el backend.
7. Implementar las 5 pantallas (modelos, servicios, componentes y rutas).

Criterio	Peso
Modelo de datos correcto (incluida tabla ternaria sin <code>@ManyToMany</code>)	20%
CRUD completo de las 4 entidades en backend + Swagger	30%
Frontend Angular con las 5 pantallas funcionando contra la API	30%
CORS, validaciones y manejo de errores (<code>ResponseEntity</code>)	10%
Gráficos y pantalla de mantenimiento	10%

⚠ Antes de borrar un socio o una actividad

Recuerda el orden de borrado que vimos en clase: primero elimina sus `suscripciones` y luego la entidad principal, o configura el borrado en cascada. Si no, MySQL rechazará el `DELETE` por la restricción de clave foránea.

👁 Pista final

Casi todo lo necesitas ya está resuelto en el proyecto `crudalumnos`: copia patrones, no inventes de cero. `Socio` ≈ `Alumno` , `Actividad` ≈ `Asignatura` , y `Suscripcion` es como `AlumnoAsignatura` pero con una FK más.